



TraceLLM: Evaluating and Exploring Large Language Models on Trace Analysis in Microservice-based Web Applications

Tong Zhou
Fudan University
Shanghai, China
tongzhou21@m.fudan.edu.cn

Xin Peng
Fudan University
Shanghai, China
pengxin@fudan.edu.cn

Jie Zhang
Samsung Electronics
(China) R&D Centre
Nanjing, China
j2.zhang@samsung.com

Chaofeng Sha
Fudan University
Shanghai, China
cfsha@fudan.edu.cn

Chenxi Zhang*
Xidian University
Xi'an, China
zhangchenxi@xidian.edu.cn

Zicheng Yuan
Fudan University
Shanghai, China
25213050062@m.fudan.edu.cn

Senyu Xie
Fudan University
Shanghai, China
24210240059@m.fudan.edu.cn

Abstract

Trace analysis is essential for understanding system behaviors, detecting anomalies, and diagnosing faults in complex microservice-based web applications. Existing trace analysis approaches face several challenges in industrial microservice-based systems, including high manual overhead, limited functionality, unfriendly interaction mechanisms, and difficulties in deployment and integration. The strong capabilities of large language models (LLMs) in natural language understanding, reasoning, and multi-task generalization provide new opportunities for a more intelligent and flexible trace analysis approach. However, the trace analysis capabilities of LLMs remain underexplored and underdeveloped. To bridge this gap, we conduct the first comprehensive evaluation on the trace analysis capabilities of LLMs. In particular, we construct the first instruction&response benchmark dataset for trace analysis, named TraceBench. It involves a wide range of trace analysis tasks, allowing us to systematically evaluate the capabilities of LLMs in this area. Experimental results show that LLMs have potential in handling trace analysis tasks, but there leaves room for improvement. To this end, we propose TraceLLM, an approach that significantly enhances the capabilities of LLMs via fine-tuning, outperforming the open-source LLMs by 34.77% on average in terms of accuracy, and outperforming the closed-source model by 21.66% in the best case. The generalization and robustness of TraceLLM are also confirmed in our experiments. To the best of our knowledge, TraceLLM is the first LLM which is specialized for handling various types of trace analysis tasks. This work provides a foundation for future research to further explore the trace analysis capabilities of LLMs.

CCS Concepts

• **Software and its engineering**; • **Computer systems organization** → **Reliability**; **Maintainability and maintenance**; • **Networks** → **Cloud computing**;

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792164>

Keywords

Microservice Architecture; Web Application; Trace Analysis; Trace; Large Language Model

ACM Reference Format:

Tong Zhou, Xin Peng, Jie Zhang, Chaofeng Sha, Chenxi Zhang, Zicheng Yuan, and Senyu Xie. 2026. TraceLLM: Evaluating and Exploring Large Language Models on Trace Analysis in Microservice-based Web Applications. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792164>

Resource Availability:

The data, models, and source code of this paper have been made publicly available at <https://doi.org/10.5281/zenodo.18326852>.

1 Introduction

Nowadays, distributed tracing [38, 37, 41] has been an important part of the infrastructures of industrial microservice-based systems. Each produced trace describes the workflow (i.e., invocation chain) of a request through service instances, and records the rich attribute information of the involved operations (i.e., service invocation) such as response code, duration time, and HTTP method, etc [53, 27]. Therefore, traces have been widely used to understand system behaviors [21], detect anomalies [52, 29, 35], and diagnose faults [50, 59, 56] in microservice-based web applications.

Visualization is the mainstream approach for trace analysis in microservice-based systems [27, 32, 4]. These approaches first automatically perform one or several predefined trace analysis tasks, such as end-to-end response time calculation [21], service call path identification [21], and trace aggregation [1]. Then present the analysis results through dashboards, enabling operation engineers to incorporate their practical experience to improve the effectiveness of downstream tasks such as detecting anomalies [57, 14, 23] and diagnosing faults [58, 21]. However, these approaches face the following challenges when applied to real-world web applications. Firstly, these approaches are typically designed for specific analysis tasks. When new analytical capabilities are required, additional modules often need to be developed, resulting in high maintenance costs. Secondly, these approaches provide limited interaction mechanisms, often relying on predefined rules or static dashboards, which places

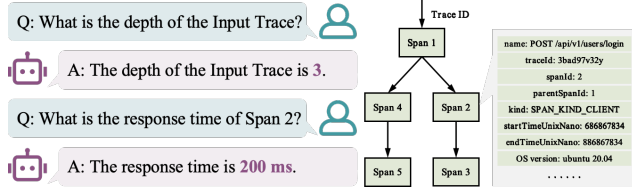


Figure 1: Example of the Interaction between Users and LLMs

high demands on the domain knowledge and experience of operation engineers. Thirdly, these approaches lack flexible interfaces and standardized deployment mechanisms, making them difficult to integrate into existing observability platforms.

With the rapid development of large language models (LLMs), their strong capabilities in natural language understanding, reasoning, and multi-task generalization have provided new opportunities for a more intelligent and flexible trace analysis approach. Ideally, LLM can act as a trace analysis expert to answer trace-related questions, Figure 1 is an example. Given a trace, which is modeled as a tree structure of spans following the OpenTelemetry [37] specification. The edges represent the caller-callee relationships between a span and its parent span. Each span records the context of a service invocation, such as service name, and start time. Through the user-friendly natural language interaction, operation engineers can obtain the structure information, such as service invocations, and the attribute information, such as end-to-end response time, which provide essential support for anomaly detection and fault diagnose. In addition, LLMs provide unified interfaces and can be encapsulated as services to integrate with existing observability tools, simplifying deployment and reducing maintenance costs.

Therefore, we conduct the first comprehensive evaluation on the trace analysis capabilities of LLMs. To support this evaluation, we construct an instruction&response benchmark dataset, named TraceBench, based on a medium-scale open-source microservice-based web application. TraceBench is the first benchmark dataset for trace analysis, and contains 38 tasks involving multiple analysis granularities and analysis objectives. For each task, we design instruction and response templates, and manually implement scripts to ensure the correctness of responses. Based on the TraceBench, we evaluate the trace analysis capabilities of open-source and closed-source LLMs, and explore the impact of different prompt engineering strategies and trace representation strategies. The experimental results show that some LLMs have certain capabilities in trace analysis. The closed-source model GPT-4o [19] achieves an accuracy of 0.734, and the best-performing open-source model achieves 0.694. This indicates that LLMs have potential in trace analysis.

To this end, we introduce fine-tuning technique to further explore the capabilities of LLMs. We propose TraceLLM, the first LLM for various trace analysis tasks. Based on different trace representation strategies, TraceLLM contains two variants: a Trace2Text-based model and a Trace2Token-based model. To evaluate the effectiveness of TraceLLM, we conduct a series of experimental studies based on TraceBench and a new dataset constructed on a small-scale open-source web application. The results show that the two variants outperform the open-source LLMs by 51.34% and 18.20% on average in terms of accuracy, and outperform the closed-source

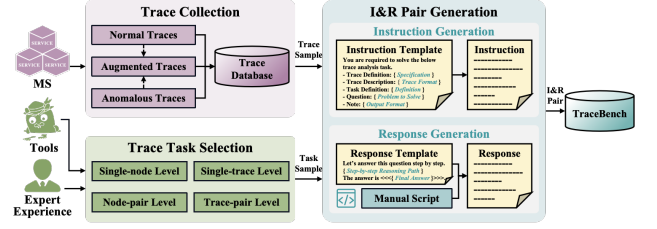


Figure 2: Overview of TraceBench Construction Process

model GPT-4o by 21.66% in the best case. In addition, TraceLLM demonstrates the generalization on unseen tasks and unseen web applications, and the robustness on traces of different lengths.

In summary, this paper makes the following contributions:

- **the first comprehensive evaluation** on trace analysis capabilities of open-source and closed-source models;
- **the first benchmark dataset**, named TraceBench, designed for evaluating and enhancing the trace analysis capabilities of LLMs;
- **the first LLM**, named TraceLLM, specialized for various trace analysis tasks, outperforms the closed-source model GPT-4o, and demonstrates generalization and robustness.

2 Dataset Construction

We construct the first instruction&response benchmark dataset for traces, named **TraceBench**. This section outlines the detailed procedures for constructing this dataset, including trace collection, trace task selection, and instruction&response pair generation. An overview of our dataset construction process is shown in Figure 2.

2.1 Trace Collection

The construction of TraceBench is based on a medium-scale benchmark microservice-based web application, TrainTicket [58, 60]. It provides typical functionalities of online train ticket booking and contains 47 services implemented in different languages. More deployment details can be found in Appendix B.1, and more details of trace dataset enhancement can be found in Appendix B.3.

In summary, TraceBench is constructed based on 13,279 traces, which can be divided into three parts according to their sources: 3,496 filtered normal traces (26.33%) collected during normal executions of TrainTicket, 6,920 filtered anomalous traces (52.11%) collected during fault injection experiments, and 2,863 augmented traces (21.56%) generated using predefined augmentation rules. The longest trace in our dataset contains more than 90 spans; each span contains at least 5 attributes and at most 18 attributes.

2.2 Trace Task Selection

To obtain diverse, representative, and practically meaningful trace analysis tasks, we organize the tasks mentioned in existing tools [42, 61, 25] and previous work [21, 1, 57]. Common observability tools such as Jaeger [25], SkyWalking [42], and Zipkin [61] provide trace analysis capabilities at multiple levels. For a single node, they support identifying caller/callee service of a specific service, and calculating response time of a span. For a single trace, they support

Table 1: Overview of Tasks in TraceBench

Task Type		Task	# Count
Granularity	Objective		
Single-node Level	Up&Down Analysis	Caller Service Identification, Caller Service Instance Identification Callee Service Identification, Callee Service Instance Identification Child Span Counting	5
	Attribute Analysis	Longest Child Span Identification Response Time Calculation, Waiting Time Calculation, Local Execution Time Calculation	4
Node-pair Level	Up&Down Analysis	Service Call Pair Existence Check, Service Instance Call Pair Existence Check Service Call Path Existence Check, Service Instance Call Path Existence Check	4
	Attribute Analysis	Response Time Comparison, Waiting Time Comparison, Local Execution Time Comparison	3
Single-trace Level	Structure Analysis	Trace Depth Calculation, Cyclic Call Detection	2
	Attribute Analysis	End-to-End Response Time Calculation, Longest Database Operation Identification Most Frequently Called Service Identification, Most Frequently Called Service Instance Identification Most Frequently Executed Operation Identification, Most Frequently Executed SQL Type Identification Most Frequently Accessed Table Identification, Most Frequently Accessed Database Identification Abnormal Response Code Operation Identification, Abnormal Response Code Span Identification	10
Trace-pair Level	Structure Analysis	Span Count Comparison, Depth Comparison Service Consistency Check, Service Instance Consistency Check, Operation Consistency Check Service Call Consistency Check, Service Instance Call Consistency Check	7
	Attribute Analysis	Response Code Comparison, Root URL Comparison, End-to-End Response Time Comparison	3

calculating end-to-end response time, and counting spans. For multiple traces, they support ranking services and operations by response time. Jaeger Compare [11] further supports comparing the structural differences between two traces. Existing visualization-based trace analysis approaches provide richer analytical capabilities to support downstream tasks, including identifying a service call path [21], identifying operations with abnormal response codes [1], and comparing the structural and attribute differences between two traces [57]. However, these tasks lack sufficient diversity. Therefore, we extend trace analysis tasks by drawing on existing tasks and the practical experience of operation engineers, covering multiple analysis granularities and objectives.

In summary, we select 38 distinct trace analysis tasks as shown in Table 1. These tasks cover common and critical requirements in trace analysis, and more details can be found in Appendix A.1.

2.3 Instruction&Response Pair Generation

We adopt an LLM-enhanced I&R pair generation approach, which includes two steps: instruction generation and response generation.

2.3.1 Instruction Generation. Firstly, we define a unique instruction template for each task. As shown in Figure 2, each instruction template includes: 1) **Task Definition**: the introduction and definition of the task; 2) **Trace Definition** and **Trace Description**: the definition and the format description of the input traces; 3) **Question**: the problem to be solved; 4) **Note**: the output format. Secondly, to ensure the diversity of instructions, we use GPT-4o [19] to rewrite the predefined instruction templates. For each task, we construct three distinct instruction templates, all of which are manually checked. Thirdly, when generating a specific instruction instance, we randomly select an instruction template and fill it with a valid question derived from a given trace or trace pair.

2.3.2 Response Generation. Firstly, we define a unique response template for each task. As shown in Figure 2, each response template consists of two components: the step-by-step reasoning path

and the final answer. The reasoning path is designed to enhance the interpretability of the model, and the final answer is the ground truth of the given trace analysis question. Secondly, to ensure the diversity of responses, we use GPT-4o [19] to rewrite the predefined response templates. For each task, we construct three distinct response templates, all of which are manually checked. Thirdly, we manually develop a Python script for each task to generate the final answer, which ensures the reliability of our evaluation. Fourthly, when generating a specific response instance, we randomly select a response template and identify the corresponding Python script based on the task type. The script takes the given trace or trace pair as input and outputs the accurate final answer.

In summary, for each trace analysis task, we first randomly sample a set of traces from our trace dataset, and then generate 800 different I&R pairs for each task based on the sampled traces. The I&R pairs from all tasks constitute our TraceBench. In total, TraceBench contains 30,400 distinct I&R pairs, involving 12,988 different traces. Some traces may be related to multiple trace analysis tasks.

3 Benchmarking LLM’s Trace Analysis Capabilities

3.1 Experiment Setup

3.1.1 Dataset. We randomly split 30% of the I&R pairs from each task in TraceBench to construct a test set, totaling 9,120 I&R pairs covering 38 tasks. This ensures the diversity and complexity of these tasks in the test set. The remaining 70% of TraceBench is used to construct the training set for fine-tuning LLMs in Section 5.

3.1.2 Prompt Engineering Strategy. We apply multiple prompt engineering techniques to evaluate the performance of LLMs, including **Zero-Shot**, **Few-Shot**, and **Program-of-Thought**. More details of the prompt engineering strategies can be found in Appendix A.2.

3.1.3 Trace Representation Strategy. We implement multiple strategies to transform structured traces into text sequences, including

Table 2: Evaluation of Different LLMs

LLM	Single-node Level			Node-pair Level			Single-trace Level			Trace-pair Level			Total
	Up&Down	Attr.	Total	Up&Down	Attr.	Total	Struct.	Attr.	Total	Struct.	Attr.	Total	
Qwen-3-8B	0.472	0.676	0.562	0.736	0.526	0.646	0.660	0.672	0.670	0.726	0.665	0.707	0.650
Llama-3-8B	0.256	0.330	0.289	0.315	0.456	0.376	0.410	0.393	0.396	0.383	0.373	0.380	0.363
DeepSeek-R1-8B	0.412	0.553	0.475	0.517	0.584	0.546	0.552	0.456	0.472	0.492	0.508	0.497	0.493
GPT-4o	0.680	0.889	0.773	0.526	0.720	0.609	0.479	0.697	0.660	0.871	0.887	0.876	0.734

Table 3: Evaluation of Different Prompt Engineering Strategies

Prompt	Single-node Level			Node-pair Level			Single-trace Level			Trace-pair Level			Total
	Up&Down	Attr.	Total	Up&Down	Attr.	Total	Struct.	Attr.	Total	Struct.	Attr.	Total	
Zero-Shot	0.472	0.676	0.562	0.736	0.526	0.646	0.660	0.672	0.670	0.726	0.665	0.707	0.650
Few-Shot	0.495	0.679	0.577	0.682	0.607	0.650	0.488	0.700	0.664	0.865	0.875	0.867	0.694
PoT	0.313	0.118	0.226	0.329	0.069	0.218	0.317	0.512	0.479	0.856	0.699	0.809	0.458

Table 4: Evaluation of Different Trace Representation Strategies

Representation	Single-node Level			Node-pair Level			Single-trace Level			Trace-pair Level			Total
	Up&Down	Attr.	Total	Up&Down	Attr.	Total	Struct.	Attr.	Total	Struct.	Attr.	Total	
Node Sequence	0.495	0.679	0.577	0.682	0.607	0.650	0.488	0.700	0.664	0.865	0.875	0.867	0.694
Adjacency Table	0.473	0.692	0.570	0.684	0.661	0.674	0.483	0.679	0.646	0.872	0.919	0.886	0.697
Edge List	0.463	0.719	0.577	0.667	0.658	0.663	0.487	0.695	0.660	0.854	0.918	0.873	0.697
Code-like Forms	0.517	0.630	0.567	0.634	0.630	0.632	0.431	0.685	0.643	0.859	0.933	0.881	0.686

Node Sequence, Adjacency Table-enhanced Node Sequence, Edge List-enhanced Node Sequence, and Code-like Forms. More details of these strategies can be found in Appendix A.3.

3.1.4 Evaluation Metrics. We evaluate the performance of LLMs using Accuracy (Acc) [47], which measures the proportion of correct final answers. The final answers in the TraceBench dataset include four types: boolean (e.g. Cyclic Call Existence), integer (e.g. Child Span Count), string (e.g. Response Time Comparison), and list (e.g. Callee Service Identification). For the first three types of answers, we use exact matching for evaluation; for list-type answers, the order of elements is ignored. As mentioned in Section 2.3.2, we execute the Python script for each task to generate ground truth answers, which ensures the reliability of our evaluation.

3.1.5 LLMs. Following existing work [49, 44, 47], experiments are conducted based on three open-source and a closed-source LLMs. The open-source LLMs include two foundation models Qwen-3-8B [39] and Llama-3-8B [34], and one reasoning model DeepSeek-R1-Distill-Llama-8B [13]. The closed-source LLM is GPT-4o [19].

3.2 Effectiveness for Trace Analysis

Table 2 shows the effectiveness evaluation results for trace analysis of different LLMs using the basic setup, that is, applying Zero-Shot prompting and representing traces as naive node sequences.

It can be seen that the powerful closed-source model GPT-4o achieves the best results in terms of Acc (0.734). Among the open-source models, only Qwen-3 achieves acceptable results (0.650), while the other two models perform poorly on trace analysis tasks.

The Acc of the worst-performing Llama-3 is even less than 0.400, indicating that the trace analysis tasks in our TraceBench dataset exceed the current capabilities of some LLMs. In terms of the analysis granularity, most models achieve better results at the node-pair level and the trace-pair level than at the single-node level and the single-trace level, respectively. This indicates that these models are more effective for qualitative analysis tasks, such as comparing which trace has longer end-to-end response time, rather than quantitative analysis tasks, such as calculating the precise end-to-end response time of a trace. In terms of the analysis objective, most models perform significantly better or slightly worse on attribute analysis tasks than on up&down analysis or structure analysis tasks. This indicates that these models are more effective at understanding and handling attribute information of traces than structure information. In addition, the open-source model Qwen-3 achieves the best or second-best results only to the closed-source GPT-4o at each analysis granularity. Therefore, to reduce the cost, we conduct the subsequent experiments based on Qwen-3.

In conclusion, the tasks in TraceBench exceed the capabilities of some LLMs, indicating that enhancing the trace analysis capabilities of LLMs is a valuable and challenging research direction.

3.3 Impact of Prompt Engineering Technique

Table 3 shows the effectiveness evaluation results for trace analysis of different prompt engineering strategies. To reduce token consumption, we transform the traces into the most naive node sequence format and set the number of examples to 1.

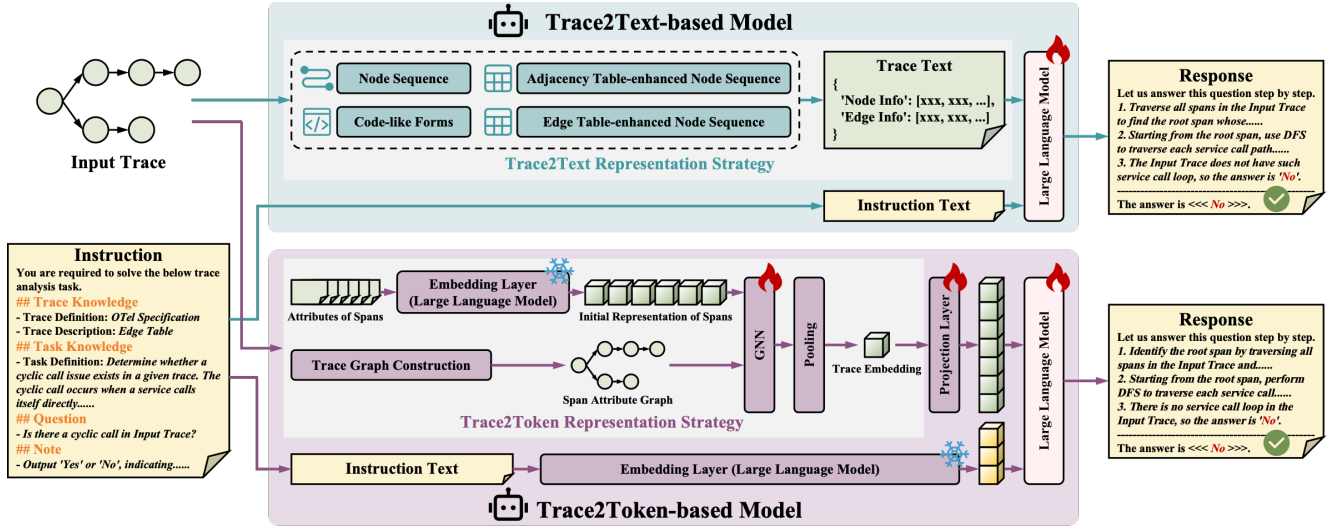


Figure 3: Overview of TraceLLM

It can be seen that the Few-Shot prompting-based trace analysis approach achieves the best result in terms of Acc (0.694), which outperforms the Zero-Shot prompting-based approach by 6.77%. And the Few-Shot prompting-based approach consistently achieves the best or near-best results in various analysis granularities and analysis objectives, indicating that trace analysis examples effectively enhance the capabilities of LLMs to understand and solve trace analysis problems. Additionally, the performance of the PoT-enhanced approach is disappointing (0.458). It achieves the second-best result at the trace-pair level (0.809), while performing poorly at other analysis granularities, with the lowest Acc of just 0.218. This is because traces have complex structure information and rich attribute information, making it difficult for LLMs to directly generate correct problem-solving code. We observe that the error cases typically fall into two categories: 1) execution failures, where the code cannot run due to issues such as incorrect number of input parameters, data type mismatch, or infinite loops; 2) semantic misunderstanding, where the code executes without error but produces incorrect results. This indicates that it is unreliable to rely solely on the code generation capabilities of LLMs to solve the trace analysis problems. Operation engineers are required to verify and modify the code of each task, which is time-consuming and impractical. Nevertheless, we still believe that the PoT-enhanced approach has potential. We will explore advanced code generation techniques in future work.

In conclusion, the experimental results demonstrate that Few-Shot is the most effective prompt engineering strategy for enhancing the trace analysis capabilities of LLMs.

3.4 Impact of Trace Representation Strategy

We use the best-performing Few-Shot strategy to guide Qwen-3 to evaluate the effectiveness of different trace representation strategies, as shown in Table 4. It can be seen that different strategies have little difference in terms of Acc, including global Acc and each fine-grained Acc. This is because the LLM, with its extensive prior knowledge and powerful text processing capabilities, can

effectively identify the structure information encoded by *spanId* and *parentspanId* within each span of the trace. The experimental results indicate that the additional explicit structure information increases the number of input tokens but have no significant improvements in Acc. The naive node sequence strategy is the best choice.

In conclusion, the most naive node sequence strategy is the best choice in terms of effectiveness and efficiency.

4 Enhancing Trace Analysis Capability via Fine-Tuning - TraceLLM

The prompt-based trace analysis approaches still leave room for improvement in terms of Acc. The main challenge is to enhance the capabilities of LLMs to capture trace features and understand trace analysis problems. Therefore, in this section, we further explore the efficacy of fine-tuning in enhancing the capabilities of LLMs. We implement a suite of models, named **TraceLLM**, based on different trace representation strategies: a **Trace2Text**-based model and a **Trace2Token**-based model, as shown in Figure 3. Each model is equipped with a dedicated training strategy to achieve the fusion of the trace knowledge and the pre-trained knowledge of LLMs. In this section, we introduce each model in detail.

4.1 Trace Representation Strategy

We design two strategies, Trace2Text and Trace2Token, to transform structured traces into representations understandable by LLMs.

4.1.1 Trace2Text. Trace2Text aims to transform traces into structured text based on a set of predefined syntactic rules. These rules can systematically describe the trace structure and the rich attributes information of each span. We design four syntactic rules to generate textual representations of traces as described in Section 3.1.3, including *Node Sequence*, *Adjacency Table-enhanced Node Sequence*, *Edge List-enhanced Node Sequence*, *Code-like Forms*.

4.1.2 Trace2Token. Trace2Token aims to use the Graph Neural Network (GNN) [48] to encode traces into embeddings. Compared

with the above Trace2Text methods, Trace2Token avoids the high computational costs of processing long textual inputs and presents a promising alternative, consisting of the following three steps:

- **Trace Graph Construction:** Given a trace t , the method constructs a span attribute graph G_t , where the node is a text string consisting of the attribute names and values of a span, and the edge represents the caller-callee relationship between two spans.
- **Node Representation Initialization:** For each node in G_t , the method first encodes the corresponding text string into a sequence of tokens using the tokenizer of a pre-trained LLM. These tokens are then embedded into a continuous space to generate their hidden embeddings. Given that the text strings of spans have different lengths, the method applies mean pooling over all token embeddings within a node to generate a fixed-length representation, which is initial representation of the node.
- **Trace Embedding Generation:** Given the initialized node representations $\hat{X}$ and the caller-callee relationships A of the trace graph G_t , the trace embedding V_t is calculated using Equation 1.

$$V_t = \text{Pool}(\text{GNN}(\hat{X}, A)) \in \mathbb{R}^{d_g} \quad (1)$$

where d_g is the hidden dimension of the GNN. The GNN function outputs the embeddings of each node in the trace graph. The Pool function outputs the trace embedding by applying a pooling operation over all node embeddings.

4.2 Model Design

In this section, we introduce the design of the Trace2Text-based model and the Trace2Token-based model in detail.

4.2.1 Trace2Text-based Model. As shown in the upper part of Figure 3, given a trace or trace pair and its corresponding instruction, the model first uses the predefined syntactic rules to transform the structured trace into a textual representation, then concatenates the transformed trace text with the instruction and feeds the combined input into the LLM, and finally outputs the LLM response, including the reasoning path and the final answer.

4.2.2 Trace2Token-based Model. As shown in the lower part of Figure 3, given a trace or trace pair and its corresponding instruction, the model first processes the structured trace. It uses the trace representation generation method described in Section 4.1.2 to generate the trace embedding V_t , and uses a projection layer, i.e., a multilayer perceptron (MLP), to align the trace embedding with the vector space of the LLM. The processed trace embedding $\hat{V}_t$ is calculated using Equation 2.

$$\hat{V}_t = \text{MLP}(V_t) \in \mathbb{R}^{d_l} \quad (2)$$

where d_l is the hidden dimension of the LLM.

After encoding the trace, the model proceeds to process the corresponding textual instruction. For each instruction I_i , the model tokenizes I_i using the frozen tokenizer of the LLM as follows:

$$I_{\text{tokens}} = \text{Tokenizer}(I_i) \quad (3)$$

The tokenizer transforms the instruction into a sequence of discrete tokens I_{tokens} in the LLM's vocabulary. These tokens are then embedded into a continuous space:

$$I_{\text{emb}} = \text{Embed}(I_{\text{tokens}}) \in \mathbb{R}^{L \times d_l} \quad (4)$$

where L is the sequence length, d_l is the hidden dimension of LLM.

Finally, the model concatenates the trace embedding $\hat{V}_t$ with the instruction embedding I_{emb} and feeds the combined embedding into the LLM to generate the response, including the reasoning path and the final answer. All three instances of LLMs in the lower part of Figure 3 use the same backbone model.

4.3 Model Training

TraceLLM applies instruction tuning to train the model simultaneously on multiple trace analysis tasks from TraceBench. This strategy aims to equip the model with comprehensive capabilities to solve a variety of trace analysis tasks by following task-specific instructions. In addition, it leverages the synergies among different tasks to enhance overall model performance. Specifically, TraceLLM uses the reasoning path and the final answer as supervision signals, and performs Supervised Fine-Tuning (SFT) with the cross-entropy loss [33]. Given a trace or trace pair t and its corresponding instruction I , the SFT loss is calculated based on the discrepancy between the model-predicted reasoning path R and final answer A , and those provided in TraceBench. The SFT loss is defined as Equation 5.

$$\mathcal{L} = - \sum_{i=1}^N \sum_{j=1}^M \log P(R_{i,j}, A_i | t_i, I_i; \theta) \quad (5)$$

where N is the number of I&R pairs in the training dataset, M is the number of reasoning paths for t_i - I_i , and θ are the parameters of the model. As shown in the upper part of Figure 3, the Trace2Text-based model requires updating the parameters of the LLM during training. As shown in the lower part of Figure 3, the Trace2Token-based model requires updating the parameters of the GNN, the projection layer, and the LLM end-to-end during training.

5 Enhancing Trace Analysis Capability via Fine-Tuning - Evaluation

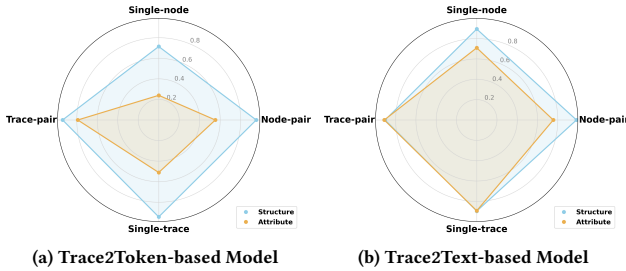
5.1 Experiment Setup

5.1.1 Dataset. We construct the following two datasets to comprehensively evaluate the effectiveness of TraceLLM:

- **Dataset- $\mathcal{A}$:** The tasks in TraceBench are divided into two groups according to whether the tasks appear during fine-tuning, which we call *seen tasks* and *unseen tasks*. We choose one task from each of the four analysis granularities of unseen tasks, while the rest as seen tasks. More details of the unseen tasks can be found in Appendix B.4. For the training set, we remove all I&R pairs corresponding to the four unseen tasks from the dataset constructed in Section 3.1.1. The remaining approximately 20,000 I&R pairs are used to construct the training set, named **Training- $\mathcal{A}$** . For the test set, we directly use the test set constructed in Section 3.1.1, named **Test- $\mathcal{A}$** , which includes both seen and unseen tasks.
- **Dataset- $\mathcal{B}$:** We construct a new small-scale benchmark dataset, named **TraceBench-OB**, based on an open-source microservice-based web application OnlineBoutique [36]. More details of trace collection can be found in Appendix B.2. This dataset contains approximately 1,000 I&R pairs covering 38 distinct tasks. These I&R pairs are used to construct a test set, named **Test- $\mathcal{B}$** .

Table 5: Effectiveness Evaluation of TraceLLM

LLM	Approach	Single-node Level			Node-pair Level			Single-trace Level			Trace-pair Level			Total
		Up&Down	Attr.	Total	Up&Down	Attr.	Total	Struct.	Attr.	Total	Struct.	Attr.	Total	
GPT-4o	Zero-Shot	0.680	0.889	0.773	0.526	0.720	0.609	0.479	0.697	0.660	0.871	0.887	0.876	0.734
Qwen-3-8B	Few-Shot	0.495	0.679	0.577	0.682	0.607	0.650	0.488	0.700	0.664	0.865	0.875	0.867	0.694
	Trace2Text	0.887	0.719	0.812	0.981	0.813	0.909	0.869	0.872	0.872	0.893	0.894	0.894	0.870
	Trace2Token	0.690	0.228	0.485	0.949	0.568	0.786	0.931	0.503	0.574	0.964	0.803	0.915	0.682
Llama-3-8B	Few-Shot	0.281	0.369	0.320	0.377	0.558	0.454	0.508	0.458	0.466	0.622	0.540	0.597	0.464
	Trace2Text	0.877	0.696	0.796	0.982	0.629	0.831	0.880	0.910	0.890	0.898	0.919	0.905	0.861
	Trace2Token	0.731	0.246	0.515	0.956	0.554	0.784	0.954	0.520	0.593	0.919	0.767	0.873	0.683
DeepSeek-R1-8B	Few-Shot	0.450	0.620	0.525	0.580	0.633	0.602	0.666	0.589	0.602	0.757	0.751	0.755	0.624
	Trace2Text	0.918	0.713	0.826	0.990	0.825	0.919	0.938	0.895	0.902	0.923	0.924	0.923	0.893
	Trace2Token	0.722	0.245	0.510	0.950	0.536	0.773	0.942	0.509	0.581	0.932	0.801	0.893	0.681

**Figure 4: Accuracy Comparison of TraceLLM**

TraceLLM, including Trace2Text-based model and Trace2Token-based model, are trained on Training- $\mathcal{A}$; all approaches, including TraceLLM and the baselines, are tested on Test- $\mathcal{A}$, except for the approaches mentioned in Section 5.3.2, which are tested on Test- $\mathcal{B}$.

5.1.2 Baseline Approaches. Details can be found in Appendix B.5.

5.1.3 Settings. Details can be found in Appendix B.6.

5.2 Effectiveness for Trace Analysis

Table 5 shows the effectiveness evaluation results for trace analysis of different approaches on different LLMs. We select the best-performing trace analysis approaches in Section 3 as baselines, including the closed-source GPT-4o with Zero-Shot strategy and three open-source LLMs with Few-Shot strategy.

It can be seen that TraceLLM significantly enhances the trace analysis capabilities of each backbone model after fine-tuning. Specifically, the Trace2Text-based model achieves an average improvement of 51.34% on three open-source models in terms of Acc, while the Trace2Token-based model achieves an average improvement of 18.20%. Notably, the Trace2Text-based model achieves the best results on all three open-source models, Qwen-3 (0.870), Llama-3 (0.861), and DeepSeek-R1 (0.893), and outperforms the powerful closed-source model GPT-4o by 19.16% on average. Meanwhile, the Trace2Token-based model achieves the second-best results on most open-source models, and performs comparable to GPT-4o, only 7.08% lower on average. This is because the introduction of fine-tuning technique helps the models better understand trace analysis problems and capture the rich attribute and structure information

in traces. These results demonstrate that our TraceLLM is effective and competitive, even compared with the closed-source models.

The Trace2Token-based model performs similarly on the three open-source LLMs. In terms of the analysis granularity, the model achieves better results at the single-trace level and the trace-pair level than at the single-node level and the node-pair level, with average Accs of 0.582 vs 0.503 and 0.893 vs 0.781, respectively. This is because the Trace2Token-based model uses GNN to encode the structure features and attribute features of the entire trace, making it more suitable for the single-trace level and trace-pair level tasks which depend on global trace representations. In contrast, the single-node level and node-pair level tasks often depend on fine-grained local representations, which may be compressed and weakened during the trace representation generation process. In addition, the model achieves significantly better results at the trace-pair level and the node-pair level than at the single-trace level and the single-node level, with average Accs of 0.893 vs 0.582 and 0.781 vs 0.503, respectively. This indicates that the model is more effective for qualitative analysis tasks than for quantitative analysis tasks. In terms of the analysis objective, the model achieves significantly better results on structure analysis tasks, including up&down analysis and structure analysis, than on attribute analysis tasks in each analysis granularity, with average improvements of 198.16%, 72.29%, 84.54%, and 18.74%, respectively. This trend is shown in Figure 4 (a), which compares the average Acc on structure analysis tasks and attribute analysis tasks at each level. More details of these attribute analysis tasks with poor performance can be found in Appendix B.7. This is due to the information compression brought by GNN and the limited capabilities of LLMs in handling computing and statistical tasks [17]. Moreover, the model achieves the best results on structure analysis tasks at both the single-trace level and the trace-pair level, further demonstrating that the Trace2Token-based model is effective for structure analysis of the entire trace.

The Trace2Text-based model achieves the best results on the reasoning model DeepSeek-R1 (0.893). In terms of the analysis granularity, the model demonstrates strong and stable performance at all levels. It performs slightly better on qualitative analysis tasks than on quantitative analysis tasks. In terms of the analysis objective, expect for the structure analysis tasks at the single-trace level and the trace-pair level, the model achieves the best results among all open-source LLMs on the remaining tasks. This is because the

Table 6: Generalization Evaluation on an Unseen Web Application

Approach	Single-node Level			Node-pair Level			Single-trace Level			Trace-pair Level			Total
	Up&Down	Attr.	Total	Up&Down	Attr.	Total	Struct.	Attr.	Total	Struct.	Attr.	Total	
Zero-Shot	0.300	0.658	0.459	0.425	0.655	0.523	0.533	0.277	0.380	0.300	0.422	0.336	0.421
Trace2Text	0.533	0.658	0.588	0.866	0.633	0.766	0.566	0.533	0.546	0.533	0.622	0.560	0.613
Trace2Token	0.547	0.300	0.437	0.808	0.533	0.690	0.650	0.000	0.260	0.533	0.557	0.550	0.502

Table 7: Generalization Evaluation on Unseen Tasks

LLM	Approach	Task-1	Task-2	Task-3	Task-4	Total
GPT-4o	Zero-Shot	0.420	0.650	0.987	0.983	0.760
DeepSeek-R1-8B	Zero-Shot	0.254	0.300	0.475	0.437	0.366
	Trace2Text	0.721	0.329	0.908	0.904	0.716
	Trace2Token	0.650	0.313	0.125	0.933	0.505

Trace2Text-based model not only preserves the rich attribute information in the trace, but also effectively extracts the structure information encoded by *spanId* and *parentspanId* in each span. In addition, the model performs better on the up&down analysis tasks at the single-node level and the node-pair level than on the attribute analysis tasks. For the structure analysis tasks and attribute analysis tasks in other granularities, the performance differences are not significant, as shown in Figure 4 (b).

In conclusion, our TraceLLM significantly enhances the capabilities of LLMs. Specifically, the Trace2Token-based model is particularly effective for structure analysis of the entire trace, and the Trace2Text-based model can effectively handle various types of tasks in different granularities. To reduce the cost, we conduct subsequent experiments based on the best-performing DeepSeek-R1.

5.3 Generalization on Unseen Tasks and Unseen Web Applications

We evaluate the generalization of TraceLLM from two aspects, *unseen tasks* and *unseen web applications*. In this experiment, we directly use the TraceLLM models trained on Training- $\mathcal{A}$ in Section 5.2, without any extra fine-tuning or retraining. For fairness, we use the Zero-Shot prompting-based approaches as baselines.

5.3.1 Evaluation on Unseen Tasks. We evaluate the generalization of TraceLLM on four unseen tasks, *Task-1* to *Task-4*. As shown in Table 7, compared with the Acc on all tasks, the Acc of TraceLLM on unseen tasks decreases, with the Trace2Text-based model decreasing by 19.82% and the Trace2Token-based model decreasing by 25.84%. Nevertheless, the performance of the Trace2Text-based model on unseen tasks is still satisfactory, which outperforms the baseline approach on the same open-source LLM by 95.63% in terms of Acc. It achieves the second-best results only to the closed-source GPT-4o on most tasks, and even outperforms GPT-4o on *Task-1* by 71.67%. The Trace2Token-based model achieves the second-best results on two structure analysis tasks, *Task-1* and *Task-4*, which outperforms the baseline approach on the same LLM by 155.91% and 113.50% respectively. This indicates that mixed-task training is effective, and TraceLLM is capable of transferring skills learned from seen tasks to specific unseen tasks. The experimental results

demonstrate that TraceLLM has a certain generalization ability on unseen tasks and can achieve competitive results compared with the advanced open-source and closed-source models.

5.3.2 Evaluation on Unseen Web Applications. We evaluate the generalization of TraceLLM on Test- $\mathcal{B}$, which is constructed on a new microservice-based web application OnlineBoutique. As shown in Table 6, compared with the seen web application, the Acc of TraceLLM on the unseen web application decreases, with the Trace2Text-based model decreasing by 31.35% and the Trace2Token-based model decreasing by 26.28%. Moreover, TraceLLM does not perform well on some types of tasks, especially the Trace2Token-based model at the single-trace level (0.260). This indicates that these tasks are inherently challenging and the training data provided to TraceLLM may be still insufficient. Nevertheless, TraceLLM still outperforms the baseline approach on the same open-source LLM (DeepSeek-R1). The Trace2Text-based model and the Trace2Token-based model are 45.61% and 19.24% higher than the baseline approach respectively in terms of Acc, and they achieve the best or second-best results on most types of tasks. The experimental results demonstrate that TraceLLM has the generalization ability on unseen web applications, but there remains room for improvement, which we leave to future work.

In conclusion, our TraceLLM shows the generalization ability on both unseen tasks and unseen web applications, and has the potential to be applied to other operation and maintenance scenarios.

6 Conclusion

In this paper, we conduct the first comprehensive evaluation on the trace analysis capabilities of LLMs, including open-source LLMs and closed-source LLMs. As a foundation for this evaluation, we construct the first instruction&response benchmark dataset for trace analysis, named TraceBench. It consists of a wide range of trace analysis tasks, involving various analysis granularities and analysis objectives. Experimental results show that LLMs have potential in handling trace analysis tasks, but there leaves room for improvement. To this end, we propose an effective approach, named TraceLLM, which significantly enhances the trace analysis capabilities of LLMs via fine-tuning. Our TraceLLM outperforms the open-source LLMs by 34.77% on average in terms of accuracy, and outperforms the closed-source model by 21.66% in the best case. The generalization and robustness of TraceLLM are also confirmed in our experiments.

Acknowledgments

The work is supported by the National Key R&D Program of China (Grant No.2024YFB4505904), and the Natural Science Basic Research Program of Shaanxi (Program No.2025JC-YBQN-851).

References

- [1] Vaastav Anand, Matheus Stolet, Thomas James Davidson, Ivan Beschastnikh, Tamara Munzner, and Jonathan Mace. 2020. Aggregate-driven trace visualizations for performance debugging. *CoRR*, abs/2010.13681.
- [2] Yushi Bai et al. 2023. Benchmarking foundation models with language-model-as-an-examiner. *Advances in Neural Information Processing Systems*, 36, 78142–78167.
- [3] Yejin Bang et al. 2023. A multitask, multilingual, multimodal evaluation of chatgpt on reasoning, hallucination, and interactivity. *arXiv preprint arXiv:2302.04023*.
- [4] André Bento, Jaime Correia, Ricardo Filipe, Filipe Araújo, and Jorge Cardoso. 2021. Automated analysis of distributed tracing: challenges and research directions. *J. Grid Comput.*
- [5] Ning Bian, Xianpei Han, Le Sun, Hongyu Lin, Yaojie Lu, Ben He, Shanshan Jiang, and Bin Dong. 2023. Chatgpt is a knowledgeable but inexperienced solver: an investigation of commonsense problem in large language models. *arXiv preprint arXiv:2303.16421*.
- [6] Shaked Brody, Uri Alon, and Eran Yahav. 2021. How attentive are graph attention networks? *arXiv preprint arXiv:2105.14491*.
- [7] Rui Cao and Qiao Wang. 2024. An evaluation of standard statistical models and llms on time series forecasting. *CoRR*, abs/2408.04867.
- [8] Yupeng Chang et al. 2024. A survey on evaluation of large language models. *ACM transactions on intelligent systems and technology*, 15, 3, 1–45.
- [9] ChaosBlade. 2025. Retrieved February 8, 2025. (2025). <https://chaosblade.io/>.
- [10] Joseph Chervenak, Harry Lieman, Miranda Blanco-Breindel, and Sangita Jindal. 2023. The promise and peril of using a large language model to obtain clinical information: chatgpt performs strongly as a fertility counseling tool with limitations. *Fertility and sterility*, 120, 3, 575–583.
- [11] Jaeger Compare. 2018. Retrieved February 8, 2025. (2018). <https://medium.com/jaegertracing/trace-comparisons-arrive-in-jaeger-1-7-a97ad5e2d05d>.
- [12] Tianyu Cui et al. 2024. Logeval: A comprehensive benchmark suite for large language models in log analysis. *CoRR*.
- [13] DeepSeek-R1-Distill-Llama-8B. 2025. Retrieved February 8, 2025. (2025). <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Llama-8B>.
- [14] Tânia Esteves, Francisco Neves, Rui Oliveira, and João Paulo. 2021. CAT: content-aware tracing and analysis for distributed systems. In *Middleware '21: 22nd International Middleware Conference, Québec City, Canada, December 6 - 10, 2021*.
- [15] LLaMA Factory. 2025. Retrieved February 8, 2025. (2025). <https://llamafactory.readthedocs.io/en/latest/>.
- [16] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2023. Talk like a graph: encoding graphs for large language models. *arXiv preprint arXiv:2310.04560*.
- [17] Riley Goodside. 2024. Retrieved February 8, 2025. (2024). <https://x.com/goodside/status/1830470374321963103>.
- [18] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. Tora: a tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*.
- [19] GPT-4o. 2024. Retrieved February 8, 2025. (2024). <https://openai.com/index/hello-gpt-4o/>.
- [20] Jiayan Guo, Lun Du, Hengyu Liu, Mengyu Zhou, Xinyi He, and Shi Han. 2023. Gpt4graph: can large language models understand graph structured data? an empirical evaluation and benchmarking. *arXiv preprint arXiv:2305.15066*.
- [21] Xiaofeng Guo, Xin Peng, Hanzhang Wang, Wanxue Li, Huai Jiang, Dan Ding, Tao Xie, and Liangfei Su. 2020. Graph-based trace analysis for microservice architecture understanding and problem diagnosis. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1387–1397.
- [22] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: low-rank adaptation of large language models. *ICLR*, 1, 2, 3.
- [23] Lexiang Huang and Timothy Zhu. 2021. Tprof: performance profiling via structural aggregation and automated analysis of distributed systems traces. In *SoCC '21: ACM Symposium on Cloud Computing, Seattle, WA, USA, November 1 - 4, 2021*.
- [24] Shaohan Huang et al. 2023. Language is not all you need: aligning perception with language models. *Advances in Neural Information Processing Systems*, 36, 72096–72109.
- [25] Jaeger. 2025. Retrieved February 8, 2025. (2025). <https://www.jaegertracing.io/>.
- [26] Ming Jin et al. 2024. Time-llm: time series forecasting by reprogramming large language models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*.
- [27] Bowen Li, Xin Peng, Qilin Xiang, Hanzhang Wang, Tao Xie, Jun Sun, and Xuanzhe Liu. 2022. Enjoy your observability: an industrial survey of microservice tracing and analysis. *Empirical Software Engineering*, 27, 1–28.
- [28] Zeyan Li et al. 2021. Practical root cause localization for microservice systems via trace analysis. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQoS)*. IEEE, 1–10.
- [29] Ping Liu et al. 2020. Unsupervised detection of microservice trace anomalies through service-level deep bayesian networks. In *2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 48–58.
- [30] Yilun Liu et al. 2024. Loglm: from task-based to instruction-based automated log analysis. *arXiv preprint arXiv:2410.09352*.
- [31] Grafana Loki. 2025. Retrieved February 8, 2025. (2025). <https://grafana.com/oss/loki/>.
- [32] Shutian Luo, Huanle Xu, Chengzhi Lu, Kejiang Ye, Guoyao Xu, Liping Zhang, Jian He, and Cheng-Zhong Xu. 2022. An in-depth study of microservice call graph and runtime performance. *IEEE Trans. Parallel Distributed Syst.*
- [33] Anqi Mao, Mehryar Mohri, and Yutao Zhong. 2023. Cross-entropy loss functions: theoretical analysis and applications. In *International conference on Machine learning*. PMLR, 23803–23828.
- [34] Meta-Llama-3-8B-Instruct. 2025. Retrieved February 8, 2025. (2025). <https://huggingface.co/meta-llama/Meta-Llama-3-8B>.
- [35] Sasho Nedelkoski, Jorge Cardoso, and Odej Kao. 2019. Anomaly detection from system tracing data using multimodal deep learning. In *2019 IEEE 12th International Conference on Cloud Computing (CLOUD)*. IEEE, 179–186.
- [36] OnlineBoutique. 2025. Retrieved February 8, 2025. (2025). <https://github.com/GoogleCloudPlatform/microservices-demo>.
- [37] OpenTelemetry. 2025. Retrieved February 8, 2025. (2025). <https://opentelemetry.io/>.
- [38] OpenTracing. 2025. Retrieved February 8, 2025. (2025). <https://opentracing.io/>.
- [39] Qwen3-8B. 2025. Retrieved February 8, 2025. (2025). <https://huggingface.co/Qwen/Qwen3-8B>.
- [40] LLaMA Factory Repo. 2025. Retrieved February 8, 2025. (2025). <https://github.com/hiyouga/LLaMA-Factory>.
- [41] Benjamin H Sigelman, Luiz André Barroso, Mike Burrows, Pat Stephenson, Manoj Plakal, Donald Beaver, Saul Jaspan, and Chandan Shanbhag. 2010. Dapper, a large-scale distributed systems tracing infrastructure.
- [42] Apache SkyWalking. 2025. Retrieved February 8, 2025. (2025). <https://skywalking.apache.org/>.
- [43] Jianheng Tang, Qifan Zhang, Yuhua Li, Nuo Chen, and Jia Li. 2024. Grapharena: evaluating and exploring large language models on graph computation. *arXiv preprint arXiv:2407.00379*.
- [44] Jianheng Tang, Qifan Zhang, Yuhua Li, Nuo Chen, and Jia Li. 2025. Grapharena: evaluating and exploring large language models on graph computation. In *The Thirteenth International Conference on Learning Representations*.
- [45] TraceLLM. 2025. Retrieved July 18, 2025. (2025). <https://doi.org/10.5281/zenodo.18326852>.
- [46] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. 2017. Graph attention networks. *stat*, 1050, 20, 10–48550.
- [47] Xilong Wang, Hao Fu, Jindong Wang, and Neil Zhenqiang Gong. 2024. Stringllm: understanding the string processing capability of large language models. *arXiv preprint arXiv:2410.01208*.
- [48] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2021. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32, 4–24.
- [49] Zhe Xie, Zeyan Li, Xiao He, Longlong Xu, Xidao Wen, Tieying Zhang, Jianjun Chen, Rui Shi, and Dan Pei. 2024. Chats: aligning time series with llms via synthetic data for enhanced understanding and reasoning. *arXiv preprint arXiv:2412.03104*.
- [50] Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. 2021. Microrank: end-to-end latency issue localization with extended spectrum analysis in microservice environments. In *Proceedings of the Web Conference 2021*, 3087–3098.
- [51] Chenxi Zhang, Zhen Dong, Xin Peng, Bicheng Zhang, and Miao Chen. 2024. Trace-based multi-dimensional root cause localization of performance issues in microservice systems. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–12.
- [52] Chenxi Zhang, Xin Peng, Chaofeng Sha, Ke Zhang, Zhenqing Fu, Xiya Wu, Qingwei Lin, and Dongmei Zhang. 2022. Deeptralog: trace-log combined microservice anomaly detection through graph-based deep learning. In *Proceedings of the 44th International Conference on Software Engineering*, 623–634.
- [53] Chenxi Zhang, Xin Peng, Tong Zhou, Chaofeng Sha, Zhenghui Yan, Yiru Chen, and Hong Yang. 2022. Tracecr: contrastive representation learning for microservice trace analysis. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1221–1232.
- [54] Ke Zhang, Chenxi Zhang, Xin Peng, and Chaofeng Sha. 2022. Putracead: trace anomaly detection with partial labels based on gnn and pu learning. In *2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 239–250.
- [55] Zihuai Zhao et al. 2024. Recommender systems in the era of large language models (llms). *IEEE Transactions on Knowledge and Data Engineering*, 36, 11, 6889–6907.

- [56] Tong Zhou, Chenxi Zhang, Xin Peng, Zhenghui Yan, Pairui Li, Jianming Liang, Haibing Zheng, Wujie Zheng, and Yuetang Deng. 2023. Tracestream: anomalous service localization based on trace stream clustering with online feedback. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 601–611.
- [57] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault analysis and debugging of microservice systems: industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering*, 47, 2, 243–260.
- [58] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2018. Fault analysis and debugging of microservice systems: industrial survey, benchmark system, and empirical study. *IEEE Transactions on Software Engineering*, 47, 2, 243–260.
- [59] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Dewei Liu, Qilin Xiang, and Chuan He. 2019. Latent error prediction and fault localization for microservice applications by learning from system trace logs. In *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 683–694.
- [60] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chenjie Xu, Chao Ji, and Wenyun Zhao. 2018. Benchmarking microservice systems for software engineering research. In *Proceedings of the 40th International Conference on Software Engineering: Companion Proceedings*, 323–324.
- [61] Zipkin. 2025. Retrieved February 8, 2025. (2025). <https://zipkin.io/>.

A More Approach Details

A.1 Details of Trace Tasks

We organize these tasks using a hierarchical classification strategy consisting of two levels.

The first level groups tasks according to the *analysis granularity*:

- **Single-node Level:** Focuses on the analysis of a single service or service instance.
- **Node-pair Level:** Focuses on the analysis of a pair of services or service instances.
- **Single-trace Level:** Focuses on the analysis of an individual trace.
- **Trace-pair Level:** Focuses on the analysis of a pair of traces.

The second level groups tasks according to the *analysis objective*:

- **Up&Down Analysis:** Focuses on the analysis of the invocation relationships between services or service instances.
- **Attribute Analysis:** Focuses on the analysis of the diverse status and metrics in traces.
- **Structure Analysis:** Focuses on the analysis of the overall structural features of traces.

A.2 Details of Prompt Engineering Strategies

We apply the following prompt engineering techniques to evaluate the performance of LLMs.

- **Zero-Shot:** The instructions are input directly to the LLMs without any additional guidance. This setting allows LLMs to autonomously determine a suitable strategy for solving the given trace analysis question. The overall prompt structure is shown in Figure 5 (a).
- **Few-Shot:** The Few-Shot prompting technique provides LLMs with a small number of input-output examples for specific trace analysis tasks. Using such examples, LLMs can better adapt to the tasks and improve generalization. Therefore, we select multiple examples for each task. The overall prompt structure is shown in Figure 5 (b).
- **Program-of-Thought:** LLMs often struggle with mathematical reasoning [44, 18], which is essential for handling complex trace analysis tasks. Inspired by the approach of writing code for

math problem-solving [18], we explore the use of Program-of-Thought (PoT) prompting technique to guide LLMs in generating executable and programmatic solutions for trace analysis tasks. This technique structures the reasoning process of LLMs with code. Therefore, it has the potential to improve the accuracy of LLMs when handling complex trace analysis tasks. The overall prompt structure is shown in Figure 5 (c).

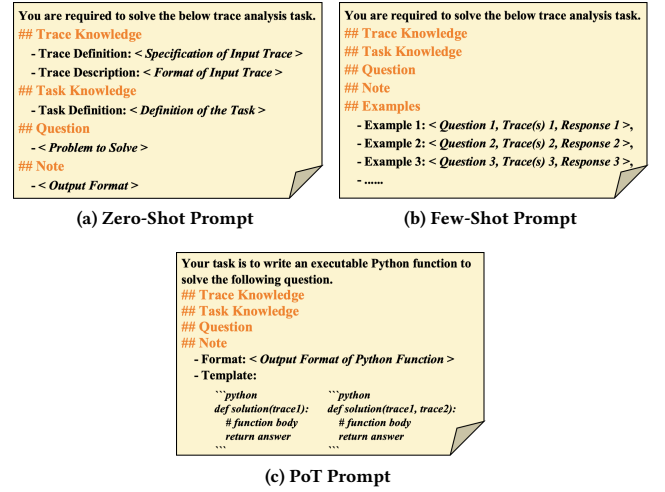


Figure 5: Overall Structure of Prompts

A.3 Details of Trace Representation Strategies

We implement the following four methods to transform structured traces into text sequences.

- **Node Sequence:** This is the most naive method. As shown in Figure 6 (a), each span is represented as a dictionary of {attribute name}-{attribute value} pairs, and a trace is represented as a sequence of span dictionaries. This method requires LLMs to autonomously reconstruct the invocation relationships within the trace based on the *spanId* and *parentSpanId* of each span.
- **Adjacency Table-enhanced Node Sequence:** The adjacency table is a widely used method for describing graph structures [16, 20]. We use an adjacency table to enumerate the child spans of each span, offering a compact way to explicitly point out the invocation relationships within a trace. As shown in Figure 6 (b), this method represents a trace using two components: a node sequence and an adjacency table.
- **Edge List-enhanced Node Sequence:** We use an edge list to detail all the invocation relationships within a trace, providing a more intuitive representation. As shown in Figure 6 (c), this method represents a trace using two components: a node sequence and an edge list.
- **Code-like Forms:** Given that LLMs demonstrate robust code comprehension capabilities [20], we explore a code-like representation, such as XML, to describe a trace. As shown in Figure 6 (d), this method transforms a trace into a highly structured XML.

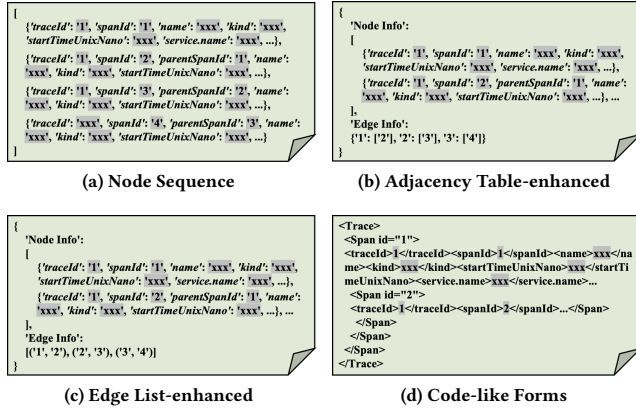


Figure 6: Trace Representation Strategies

B More Experiment Details

B.1 Details of Trace Collection on TrainTicket

We deploy TrainTicket on a Kubernetes cluster consisting of 6 virtual machines. Each machine is equipped with an 8-core Intel Xeon 3.0Hz CPU and 24GB RAM, running CentOS 7.7. To simulate real-world deployment scenarios, some services are deployed as multiple instances, with a total of 90 instances. We use OpenTelemetry [37] as the distributed tracing framework to collect traces and use Grafana Tempo [31] as the distributed tracing backend.

We use the load generator provided by TrainTicket [58, 60] to execute automated Python scripts to simulate different user requests to generate traces, covering a total of 58 different user requests.

B.2 Details of Trace Collection on OnlineBoutique

OnlineBoutique is a web-based e-commerce application and contains 11 services implemented in different languages. We deploy OnlineBoutique using the same specifications and environment as those used for TrainTicket in Section 2.1. In total, we collect and filter out 1,855 traces from OnlineBoutique and follow the same procedure mentioned in Section 2 to construct the TraceBench-OB dataset.

B.3 Details of Trace Dataset Enhancement

To ensure the diversity of traces in TraceBench, we perform the following steps to process and enhance the trace dataset. Firstly, during the trace generation process, we use fault injection to obtain anomalous traces following the previous work [51, 28, 29, 50, 59]. Specifically, we adopt 5 fault types as shown in Table 8 and use the widely used chaos engineering tool ChaosBlade [9] to inject each fault to different services to produce different fault instances. Secondly, we filter out redundant traces in normal and anomalous sets based on their attribute information and structure information, including the number of spans, and root URL, etc. Thirdly, based on the filtered traces, we further enhance trace diversity by applying a set of predefined trace augmentation rules following the previous work [53], including adding/removing nodes, adding edges, and injecting response code/response time/span order errors. Specifically,

we first sample a subset of the filtered traces, including normal traces and anomalous traces, and then apply a randomly selected augmentation rule to each sampled trace.

Table 8: Fault Types in the TraceBench Dataset

Fault Type	Description
Network Delay	Network jam increases communication time.
CPU Exhausted	CPU exhaustion increases invocation processing time.
API Delay	API bugs or traffic issues increase its invocation time.
Slow SQL	Table design or SQL bugs increase query processing time.
Producer Delay	Excessive concurrency delays message production.

B.4 Details of Unseen Tasks

The four unseen tasks include: 1) *Task-1*: Identification of the caller of a service instance, 2) *Task-2*: Comparison of the waiting time of spans, 3) *Task-3*: Identification of frequently accessed databases, 4) *Task-4*: Consistency comparison of service instances involved in traces. They involve multiple analysis objectives, namely Up&Down, Attribute, Attribute, and Structure, and multiple answer types, namely list, string, string, boolean. The details of these four unseen tasks can be found in our replication package [45].

B.5 Details of Baseline Approaches

We compare TraceLLM with the prompt-based approaches in Section 3. Specifically, we select the closed-source model GPT-4o [19] and three open-source models Qwen-3-8B [39], Llama-3-8B [34], and DeepSeek-R1-Distill-Llama-8B [13] enhanced with prompts as baselines. For the prompt engineering strategy, we select the appropriate prompt technique according to the specific experimental settings. For the trace representation strategy, since different representations show no significant difference in terms of Acc in Section 3, we use the most naive node sequence.

B.6 Details of Implementation and Settings

We implement TraceLLM in Python 3.10.16, PyTorch 2.6.0, and Pytorch Geometric 2.6.1 (for graph neural network learning). For the Trace2Text-based model, the code is built upon LLaMA-Factory [15, 40]; LoRA [22] is used to fine-tuning the model; learning rate and epochs are set to $1e^{-5}$ and 3, respectively; the most naive node sequence format is used to transform traces into textual representations. For the Trace2Token-based model, GAT [46, 6] is used to encode traces into embeddings; LoRA [22] is used to fine-tuning the model; learning rate and epochs are set to $1e^{-5}$ and 15, respectively; the number of layers, hidden dimension size, and pooling strategy of GAT are set to 2, 1024, and add pooling. All the experimental studies are conducted on a Linux cluster with NVIDIA 8*A800 GPUs, and running Ubuntu 20.04.

B.7 Details of Tasks with Poor Performance

These attribute analysis tasks which limit the performance of the model can be further divided into four categories: *computation tasks*, *ranking tasks*, *comparison tasks*, and *component identification tasks*. As shown in Table 9, the model performs poorly on computation tasks and ranking tasks, with average Accs of only 0.091 and 0.484.

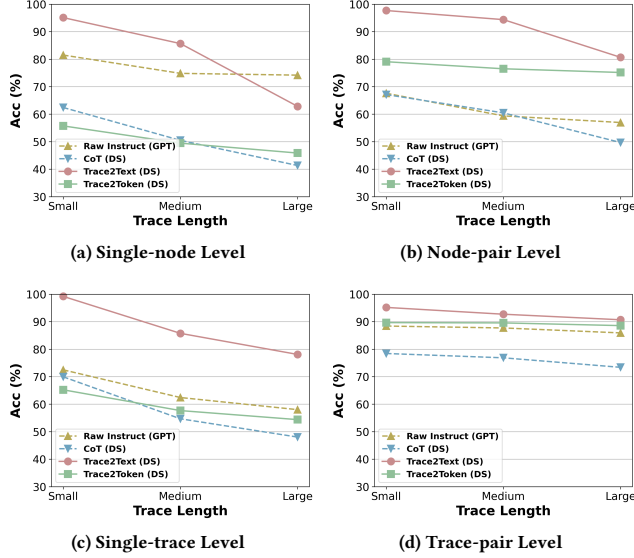
Table 9: Evaluation on Attribute Analysis Tasks

LLM	Computation	Ranking	Comparison	Identification
Qwen-3-8B	0.076	0.476	0.685	0.910
Llama-3-8B	0.100	0.495	0.660	0.912
DeepSeek-R1-8B	0.097	0.482	0.668	0.908

B.8 Impact of Trace Length

We collect the length of each trace in Test- $\mathcal{A}$, measured by the number of spans, and compute the tertiles of trace lengths. Based on these thresholds, all I&R pairs are divided into three groups: 1) *Small*: $\text{trace length} \leq 11$, 2) *Medium*: $11 < \text{trace length} \leq 26$, 3) *Large*: $\text{trace length} > 26$. As shown in Figure 7, as the trace length increases, TraceLLM and all baseline approaches consistently experience a decline in Acc across different analysis granularities. This aligns with our expectations, given that more complex traces inherently pose more challenging reasoning tasks for LLMs. Notably, despite the increased difficulty, TraceLLM consistently outperforms the baseline approach on the same open-source LLM, and performs better or slightly worse than the closed-source GPT-4o. The experimental results demonstrate that TraceLLM has the robustness in handling traces of different lengths.

In conclusion, our TraceLLM shows the robustness in handling traces of different lengths and shows the advantages at different analysis granularities.

**Figure 7: Impact of Trace Length**

C Threats to Validity

The internal threats in our studies are mainly from the sub-optimal performance. We use the default settings during fine-tuning without adjusting hyperparameters, which may result in sub-optimal performance. Nevertheless, our TraceLLM still achieves competitive

and satisfactory results. In addition, the choice of underlying foundation models may also be sub-optimal. We choose the LLMs which are widely used in related research [43, 47, 49, 30] as the underlying foundation models. As optimal settings vary by deployment environment, we leave further tuning of settings to practitioners.

The external threats in our studies are mainly from the representativeness of the benchmark systems and the trace analysis tasks. We have only conducted experiments on two open-source benchmark systems, namely TrainTicket and OnlineBoutique, as there are no public available datasets from industrial systems. Although TrainTicket is one of the largest open-source microservice-based web applications and is widely used in existing research [53, 54, 56, 52], there is still a gap between its scale and industrial large-scale microservice architectures. Moreover, although we design trace analysis tasks based on existing tools [25, 11, 42, 61] and expert experience, there may still be a gap between our tasks and the tasks in industrial scenarios.

D Related Work

Trace Analysis. Trace analysis is essential for operation engineers to understand system behaviors [21], detect anomalies [52, 29, 35], and diagnose faults [50, 59, 56] in a microservice-based web application. Existing research highlights that trace analysis remains largely reliant on traditional methods such as visualization and rule-based techniques, despite growing academic interest [27, 32, 4]. Although these methods improve the efficiency and effectiveness of the downstream tasks [57, 14, 23, 21], they are less automated and still require considerable manual participation. Due to the large amount and high complexity of traces, manual analysis is impractical in large-scale industrial microservice architectures. Therefore, we propose an automated LLM-based trace analysis approach to assist operation engineers in analyzing traces more efficiently.

LLM Evaluation. Given the black-box nature of LLMs, comprehensive evaluation is essential for understanding their capabilities. So far, the evaluation for LLMs has extended across a wide range [8], covering traditional NLP tasks [2, 3, 5] and various domain-specific downstream applications [10, 55, 24]. Specifically in observability data analysis domain, current evaluation focuses primarily on time series analysis [49, 7, 26] and log analysis [30, 12]. Compared with time series and logs, traces contain complex structure information and rich attribute information, which makes them challenging for LLMs to understand directly. The trace analysis capabilities of LLMs remain underexplored and underdeveloped. Our work bridges this gap by constructing the first trace analysis benchmark dataset for evaluating the capabilities of LLMs.